Package 'phylopomp'

June 27, 2026

Contents

as.data.frame *Coerce to a Data Frame*

Description

Functions to coerce an object to a data frame.

Usage

```
## S3 method for class 'gplin'
as.data.frame(x, ...)
```

Arguments

x	any R object.
...	additional arguments to be passed to or from methods.

Details

An object of class 'gplin' is coerced to a data frame by means of `as.data.frame`.

bdei *Linear birth-death with exposed and infectious classes*

Description

Two-deme linear birth-death process with exposed (incubating) and infectious lineages. Transmission creates exposed offspring from infectious donors; exposed lineages progress to infectious. Only infectious lineages are subject to sampling and sampling is destructive. Identical to the BDEI model of Voznica et al. (2022, https://doi.org/10.1038/s41467-022-31511-0), albeit with a different parameterization.

Usage

```
runBDEI(
  time,
  t0 = 0,
  sigma = 1/7,
  lambda = 7/9,
  mu = 1/6,
  chi = 1/6,
  pop = 1,
  E0 = 0,
  I0 = 1
)

continueBDEI(object, time, sigma = NA, lambda = NA, mu = NA, chi = NA)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
sigma	per capita progression rate (exposed to infectious)
lambda	per capita reproduction rate
mu	per capita removal rate (of infectious lineages)
chi	(destructive) sampling rate
pop	initial population size
E0	initial fraction of exposed lineages
I0	initial fraction of infectious lineages
object	a previously computed simulation

Value

runBDEI and continueBDEI return objects of class 'gpsim' with 'model' attribute "BDEI".

References

J. Voznica, A. Zhukova, V. Boskova, E. Saulnier, F. Lemoine, M. Moslonka-Lefebvre, O. Gascuel. Deep learning from phylogenies to uncover the epidemiological dynamics of outbreaks. *Nature Communications* **13**, 3896, 2022. doi:10.1038/s41467022315110.

See Also

More example genealogy processes: bdss, lbdp, mers, moran, s2i2r2, seir, si2r, siir, simulate(), sir, strains, twospecies, twoundead

Examples

```
library(phylopomp)

runBDEI(time=10, pop=2, E0=0.5, I0=0.5) |>
  plot()
```

---

bdss *Linear birth-death with superspreading*

Description

Two-deme linear birth-death process with heterogeneous per-lineage transmission rates. Identical to the BDSS model of Voznica et al. (2022, https://doi.org/10.1038/s41467-022-31511-0), though the parameterization differs.

Usage

```
runBDSS(
  time,
  t0 = 0,
  lambda_nn = 0.875,
  lambda_ns = 0.125,
  lambda_sn = 5.5,
  lambda_ss = 0.75,
  mu = 0.5,
  chi = 0.5,
  pop = 1,
  N0 = 1,
  S0 = 0
)

continueBDSS(
  object,
  time,
  lambda_nn = NA,
  lambda_ns = NA,
  lambda_sn = NA,
  lambda_ss = NA,
  mu = NA,
  chi = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
lambda_nn	normal-to-normal transmission rate
lambda_ns	normal-to-superspreader transmission rate
lambda_sn	superspreader-to-normal transmission rate
lambda_ss	superspreader-to-superspreader transmission rate
mu	per capita removal (death or recovery) rate
chi	per capita destructive sampling rate
pop	initial population size
N0	initial fraction of normal spreaders
S0	initial fraction of superspreaders
object	a previously computed simulation

Value

runBDSS and continueBDSS return objects of class ‘gpsim’ with ‘model’ attribute “BDSS”.

See Also

More example genealogy processes: bdei, lbdp, mers, moran, s2i2r2, seir, si2r, siir, simulate(), sir, strains, twospecies, twoundead

---

curtail *Curtail a genealogy to the given time*

Description

Discards all nodes beyond the given time.

Usage

```
curtail(object, time = NA, t0 = NA)
```

Arguments

object	gpsim object.
t0, time	numeric scalars; determine the time interval for curtailed genealogy

Value

a genealogy object: the curtailed version of the input genealogy.

Examples

```
library(ggplot2)

simulate("SIIR",time=5) -> x

plot_grid(
  x |>
    plot(prune=FALSE,points=TRUE),
  x |>
    curtail(time=3) |>
    plot(prune=FALSE,points=TRUE)+
    expand_limits(x=5),
  ncol=1,align="h",axis="tblr"
)

plot_grid(
  x |>
    plot(prune=TRUE,points=TRUE)+
    geom_vline(xintercept=3),
  x |> curtail(time=3) |>
    plot(prune=TRUE,points=TRUE)+
    geom_vline(xintercept=3)+
    expand_limits(x=5),
  ncol=1,align="h",axis="tblr"
)
```

`diagram` *Genealogy process diagram*

Description

Produces a diagram of the genealogy process state.

Usage

```
diagram(
  object,
  prune = TRUE,
  obscure = TRUE,
  m = NULL,
  n = NULL,
  ...,
  digits = 1,
  palette = scales::hue_pal(l = 80, c = 20, h = c(220, 580))
)

## S3 method for class 'gpdiag'
print(x, newpage = is.null(vp), vp = NULL, ...)
```

Arguments

`object`	`gpsim` object.
`prune`	logical; prune the genealogy?
`obscure`	logical; obscure the demes?
`m`	width of plotting window, in nodes. By default, the nodes will be adjusted in width to fit the window.
`n`	height of the pockets, in balls. By default, the balls will be adjusted in size to fit the space available.
`...`	other arguments, ignored.
`digits`	non-negative integer; number of decimal digits to print in the node time
`palette`	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.
`x`	object of class 'gpdiag'
`newpage`	draw new empty page first?
`vp`	viewport to draw plot in

Value

A **grid** graphics object (`grob`), invisibly.

Examples

```
runSIR(Beta=3,gamma=0.1,psi=0.2,S0=100,I0=5,R0=0,time=2,t0=0) -> x
plot(x,points=TRUE,prune=FALSE)
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))
```

---

gendat *Genealogy as a data frame*

Description

Converts a given genealogy to a data frame.

Usage

```
gendat(object, obscure = TRUE)
```

Arguments

object	a 'gpgen' object.
obscure	logical; obscure the demes?

Value

A list of objects containing the information pertinent for filtering.

---

geneal *Bare genealogy*

Description

Extracts the bare genealogy from a Markov genealogy process simulation

Usage

```
geneal(object)
```

Arguments

object	a 'gpgen' object, possibly with 'model' attribute.

Value

A bare genealogy object.

geneal_scale *Rescale and/or shift time.*

Description

Time-rescaling and/or shifting.

Usage

```
geneal_scale(x, scale, origin = 0)
```

Arguments

x	genealogy to be scaled
scale, origin	scalars

Details

A linear transformation is applied to the genealogy's time-scale. Specifically, if t is the old time and t' the new time, then $t' = a(t - b)$ where $a =$`scale` and $b =$`origin`.

Value

modified genealogy

Examples

```
library(ggplot2)

runSEIR(t0=0,time=3,pop=200,E0=0.02,I0=0) -> x
geneal_scale(x,365,3) -> y
plot_grid(plot(x),plot(y),ncol=1)
```

geneal_sum *Combine genealogies*

Description

A summing operator for genealogies.

Usage

```
## S3 method for class 'gpgen'
x + y

geneal_sum(...)
```

Arguments

x, y, ...	genealogies or lists of genealogies to be summed.

Details

The sum of genealogies is defined to be the union of all the lineages, restricted to the common time-interval, i.e., the intersection of all the individual time-intervals.

Value

combined genealogy.

Examples

```
library(ggplot2)

runSEIR(t0=-1,time=2,E0=0,I0=0.03) -> x
runSEIR(t0=0,time=3,E0=0.02,I0=0) -> y
runSEIR(t0=1,time=4,E0=0.03,I0=0.02) -> z

plot_grid(
  plot_grid(
    plotlist=lapply(
      list(x,y,z),
      plot,
      points=TRUE,ladderize=FALSE,
      legend.position="none"
    ) |>
      lapply(\(.).+geom_vline(xintercept=c(-1,0,1,2,3,4))),
    labels=c("x","y","z"),
    ncol=1,align="v",axis="bltr"
  ),
  plot_grid(
    plotlist=lapply(
      list(x+y,y+z,x+y+z),
      plot,
      points=TRUE,ladderize=FALSE,
      legend.position="none"
    ) |>
      lapply(\(.).+geom_vline(xintercept=c(-1,0,1,2,3,4))),
    labels=c("x+y","y+z","x+y+z"),
    ncol=1,align="h",axis="bltr"
  )+
    geom_vline(xintercept=c(0,1,2,3,4)),
  nrow=1
)
```

genealogy diagram internals
Diagramming internals

Description

Facilities to produce diagrammatic representations of genealogy process states.

Usage

```
genealogyGrob(object, m = NULL, n = NULL, vp = NULL, palette, ...)

nodeGrob(object, digits = 1, palette, n = NULL, vp = NULL)

pocketGrob(object, n, vp = NULL)

ballGrob(object, vp = NULL)

resizingTextGrob(..., vp = NULL)

## S3 method for class 'resizingTextGrob'
drawDetails(x, recording = TRUE)

## S3 method for class 'resizingTextGrob'
preDrawDetails(x)

## S3 method for class 'resizingTextGrob'
postDrawDetails(x)

## S3 method for class 'ballGrob'
drawDetails(x, recording = TRUE)

## S3 method for class 'ballGrob'
preDrawDetails(x)

## S3 method for class 'ballGrob'
postDrawDetails(x)

## S3 method for class 'gpsim'
print(x, ...)

## S3 method for class 'gpgen'
print(x, ...)

## S3 method for class 'gpyaml'
print(x, ...)
```

Arguments

object	list; pocket structure
m	width of plotting window, in nodes. By default, the nodes will be adjusted in width to fit the window.
n	length of longest genealogy
vp	viewport to draw plot in
palette	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.
...	arguments to be passed to textGrob.
digits	non-negative integer; number of decimal digits to print in the node time
x	an object of class 'grob' or NULL
recording	A logical value indicating whether a grob is being added to the display list or redrawn from the display list.

Details

Code for the resizing text adapted from a blog post by Mark Heckmann (https://ryouready.wordpress.com/2012/08/01/creating
a-text-grob-that-automatically-adjusts-to-viewport-size/).

---

getInfo *getInfo*

Description

Retrieve information from genealogy process simulation

Usage

```
getInfo(
  object,
  prune = TRUE,
  obscure = TRUE,
  extended = TRUE,
  t0 = FALSE,
  time = FALSE,
  structure = FALSE,
  yaml = FALSE,
  ndeme = FALSE,
  lineages = FALSE,
  newick = FALSE,
  nsample = FALSE,
  nroot = FALSE,
  genealogy = FALSE,
  gendat = FALSE
)
```

Arguments

`object`	`gpsim` object.
`prune`	logical; prune the genealogy?
`obscure`	logical; obscure the demes?
`extended`	logical; return extended-Newick format?
`t0`	logical; return the zero-time?
`time`	logical; return the current time?
`structure`	logical; return the structure in R list format?
`yaml`	logical; return the structure in YAML format?
`ndeme`	logical; return the number of demes?
`lineages`	logical; return the lineage-count function?
`newick`	logical; return a Newick-format description of the tree?
`nsample`	logical; return the number of samples?
`nroot`	logical; return the number of roots?
`genealogy`	logical; return the lineage-traced genealogy?
`gendat`	logical; return the data-frame format?

Value

A list containing the requested elements, including any or all of:

t0 the initial time (a numeric scalar)

time the final time (a numeric scalar)

ndeme the number of demes (an integer)

nsample the number of samples (an integer)

nroot the number of roots (an integer)

newick the genealogical tree, in Newick format (extended-Newick if `extended=TRUE`)

yaml the state of the genealogy process in YAML format

structure the state of the genealogy process in R list format

lineages a `tibble` containing the lineage count function through time

gendat a `tibble` containing the (obscured) genealogy in a data-frame format

genealogy the lineage-traced genealogy (as a raw vector)

Examples

```
simulate("SIIR",time=3,psi1=1,psi2=0) |>
  simulate(Beta1=2,gamma=2,time=10,psi1=10,psi2=1) |>
  plot()

runSIIR(Beta1=10,Beta2=8,
  pop=220,S_0=200,I1_0=10,I2_0=8,R_0=0,time=0,t0=-1) |>
  simulate(psi1=10,time=2) |>
```

```
  plot(points=TRUE,obscure=FALSE)

simulate("SIIR",Beta1=2,Beta2=50,gamma=1,psi1=2,
  pop=322,S_0=300,I1_0=20,I2_0=2,time=5) |>
  lineages() |>
  plot()
```

---

lbdp *Linear birth-death-sampling model*

Description

The genealogy process induced by a simple linear birth-death process with constant-rate sampling.

Usage

```
runLBDP(time, t0 = 0, lambda = 2, mu = 1, psi = 1, chi = 0, n0 = 5)

continueLBDP(object, time, lambda = NA, mu = NA, psi = NA, chi = NA)

lbdp_pomp(x, lambda, mu, psi, chi = 0, n0 = 1)

lbdp_exact(x, lambda, mu, psi, chi = 0, n0 = 1)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
lambda	per capita birth rate
mu	per capita death rate
psi	per capita non-destructive sampling rate
chi	per capita destructive sampling rate
n0	population size at time t0
object	a previously computed simulation
x	genealogy in **phylopomp** format (i.e., an object that inherits from 'gpgen').

Details

lbdp_pomp constructs a **pomp** object containing a given set of data and a linear birth-death-sampling process.

lbdp_exact gives the exact log likelihood of a linear birth-death process with (optionally destructive) sampling, conditioned on the population size at time 0.

Value

`runLBDP` and `continueLBDP` return objects of class 'gpsim' with 'model' attribute "LBDP".

`lbdp_exact` returns the log likelihood of the genealogy. Note that the time since the most recent sample is informative.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:10.1016/j.tpb.2021.11.003.

T. Stadler. Sampling-through-time in birth-death trees. *Journal of Theoretical Biology* **267**, 396–404, 2010. doi:10.1016/j.jtbi.2010.09.010.

T. Stadler. Sampling-through-time in birth-death trees. *Journal of Theoretical Biology* **267**, 396–404, 2010. doi:10.1016/j.jtbi.2010.09.010.

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

See Also

More example genealogy processes: `bdei`, `bdss`, `mers`, `moran`, `s2i2r2`, `seir`, `si2r`, `siir`, `simulate()`, `sir`, `strains`, `twospecies`, `twoundead`

Examples

```
simulate("LBDP",time=4) |> plot(points=TRUE)

simulate("LBDP",lambda=2,mu=1,psi=3,n0=1,time=1) |>
  simulate(time=10,lambda=1) |>
  plot()

simulate("LBDP",time=4) |>
  lineages() |>
  plot()
```

---

lineages *Lineage-count function*

Description

Lineage-counts, saturations, and event-codes.

Usage

```
lineages(object, prune = TRUE, obscure = TRUE)

## S3 method for class 'gplin'
plot(x, ..., palette = scales::hue_pal(l = 30, h = c(220, 580)))
```

Arguments

object	gpsim object.
prune	logical; prune the genealogy?
obscure	logical; obscure the demes?
x	object of class 'gpgen'
...	passed to theme.
palette	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.

Details

This function extracts from the specified genealogy several important time-varying quantities. These include:

lineages number of lineages through time

saturation the number of lineages emerging from the event

event_type an integer coding the type of event

If the genealogy has been obscured (the default), the number in the lineages returned is the total number of lineages present at the specified time and the saturation is the total saturation. If the genealogy has not been obscured (obscure = FALSE), the deme-specific data are returned. In this case, the deme column specifies the pertinent deme.

The event types are:

0 no event,

-1 a root,

1 a sample event,

2 a non-sample event,

3 the end of the time interval, which may or may not coincide with the latest tip of the genealogy.

plot applied to the data frame produced by lineages yields a lineage-through-time plot.

Value

A tibble containing information about the genealogy. See Details for specifics. The tibble returned by lineages has a plot method.

Examples

```
pal <- c("#000000","#00274c","#ffcb05")

simulate("SIIR",time=3,psi1=1,psi2=1) -> x
plot_grid(
  x |> plot(palette=pal),
  x |> lineages() |> plot(palette=pal),
  x |> plot(obscure=FALSE,palette=pal),
  x |> lineages(obscure=FALSE) |>
    plot(palette=pal,legend.position=c(0.8,0.9)),
  align="v",axis="b",
  ncol=2,byrow=FALSE
)
```

---

mers

Two-host infection model with spillover and demography. Hosts are culled upon sampling.

Description

The population is structured by infection progression and host species.

Usage

```
runMERS(
  time,
  t0 = 0,
  Beta_cc = 4,
  Beta_ch = 0,
  Beta_hc = 0,
  Beta_hh = 4,
  gamma_c = 1,
  gamma_h = 1,
  chi_c = 1,
  chi_h = 0,
  Bc = 0,
  Bh = 0,
  Sc0 = 1,
  Sh0 = 1,
  Ic0 = 0.01,
  Ih0 = 0,
  Nc = 10000,
  Nh = 10000
)

continueMERS(
  object,
```

```
  time,
  Beta_cc = NA,
  Beta_ch = NA,
  Beta_hc = NA,
  Beta_hh = NA,
  gamma_c = NA,
  gamma_h = NA,
  chi_c = NA,
  chi_h = NA,
  Bc = NA,
  Bh = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta_cc	transmission rate within camels
Beta_ch	transmission from human to camel
Beta_hc	transmission from camel to human
Beta_hh	transmission rate within humans
gamma_c	removal rate in camels
gamma_h	removal rate in humans
chi_c	per capita destructive sampling rate for camels
chi_h	per capita destructive sampling rate for humans
Bc	gross birth rate for camels
Bh	gross birth rate for humans
Sc0	initial susceptible fraction of camel population
Sh0	initial susceptible fraction of human population
Ic0	initial infected fraction of camel population
Ih0	initial infected fraction of human population
Nc	camel population size
Nh	human population size
object	a previously computed simulation

Value

runMERS and continueMERS return objects of class 'gpsim' with 'model' attribute "MERS".

See Also

More example genealogy processes: bdei, bdss, lbdp, moran, s2i2r2, seir, si2r, siir, simulate(), sir, strains, twospecies, twoundead

`moran` *The classical Moran model*

Description

The Markov genealogy process induced by the classical Moran process, in which birth/death events occur at a constant rate and the population size remains constant.

Usage

```
runMoran(time, t0 = 0, mu = 1, psi = 1, n = 100)

continueMoran(object, time, mu = NA, psi = NA)

moran_exact(x, n = 100, mu = 1, psi = 1)
```

Arguments

`time`	end timepoint of simulation
`t0`	beginning timepoint of simulation
`mu`	per capita event rate
`psi`	per capita sampling rate
`n`	population size
`object`	a previously computed simulation
`x`	genealogy in **phylopomp** format (i.e., an object that inherits from 'gpgen').

Details

`moran_exact` gives the exact log likelihood of a genealogy under the uniformly-sampled Moran process.

Value

`runMoran` and `continueMoran` return objects of class 'gpsim' with 'model' attribute "Moran".

`moran_exact` returns the log likelihood of the genealogy.

References

P.A.P. Moran. Random processes in genetics. *Mathematical Proceedings of the Cambridge Philosophical Society* **54**, 60–71, 1958. doi:10.1017/s0305004100033193.

See Also

More example genealogy processes: `bdei`, `bdss`, `lbdp`, `mers`, `s2i2r2`, `seir`, `si2r`, `siir`, `simulate()`, `sir`, `strains`, `twospecies`, `twoundead`

newick *Newick output*

Description

Extract a Newick-format description of a genealogy.

Usage

```
newick(object, prune = TRUE, obscure = TRUE, extended = TRUE)
```

Arguments

object	gpsim object.
prune	logical; prune the genealogy?
obscure	logical; obscure the demes?
extended	logical; if TRUE, an extended-Newick format is used. See Details.

Details

In the extended-Newick format, metadata tags of the form [&&PhyloPOMP ...] are inserted. These contain information regarding node-type and deme. See below for more information.

If extended = FALSE, then the string returned is in ordinary Newick format. Deme and node-type information is discarded. Every sample becomes a tip (inline samples lie on the ends of branches of zero length). Note that extended = FALSE implies both prune = TRUE and obscure = TRUE.

Value

A string in (possibly extended) Newick format.

Metadata tags

Metadata tags are of the form

```
[&&PhyloPOMP type={node|sample|extant|root|migration|branch} deme=<integer>].
```

Types 'branch', 'node', 'migration', and 'root' are equivalent on input to parse_newick; 'type=node' is used in output of newick. Note that 0 is reserved for the "undeme", i.e., unspecified or unknown deme.

Examples

```
simulate("SIIR",time=0.3,sigma12=1) -> x

x |> plot(prune=FALSE,obscure=FALSE)

x |> newick(prune=FALSE,obscure=FALSE)

x |> newick()

x |> newick(extended=FALSE)

x |>
  newick() |>
  parse_newick(t0=0)

x |>
  newick(prune=FALSE,obscure=FALSE) |>
  parse_newick(t0=0)

x |>
  newick(extended=FALSE) |>
  parse_newick(t0=0)

"" |> parse_newick() |> newick()

";" |> parse_newick() |> newick()

"((:1,:1):1,:1):1;" |> parse_newick() |> newick()
```

---

parse_newick *parse a Newick string*

Description

Parses a Newick description and returns a binary version of the genealogy.

Usage

```
parse_newick(x, t0 = 0, time = NA)
```

Arguments

x	character; the Newick description. See Details for specifics.
t0	numeric; the root time.
time	numeric; the current or final time.

Details

parse_newick parses a string containing information in a (possibly extended) Newick format and returns a **phylopomp** genealogy.

The extension allows metadata (enclosed by square brackets). It ignores metadata unless they are specifically flagged as PhyloPOMP metadata, i.e.: [&&PhyloPOMP ...]. PhyloPOMP metadata can be used to encode the node type (i.e., type=sample, type=extant, type=node) and/or the deme (given as an integer, e.g., deme=1). See below for more information.

Value

An object of class "gpgen".

Metadata tags

Metadata tags are of the form

[&&PhyloPOMP type={node|sample|extant|root|migration|branch} deme=<integer>].

Types 'branch', 'node', 'migration', and 'root' are equivalent on input to parse_newick; 'type=node' is used in output of newick. Note that 0 is reserved for the "undeme", i.e., unspecified or unknown deme.

Examples

```
simulate("SIIR",time=0.3,sigma12=1) -> x

x |> plot(prune=FALSE,obscure=FALSE)

x |> newick(prune=FALSE,obscure=FALSE)

x |> newick()

x |> newick(extended=FALSE)

x |>
  newick() |>
  parse_newick(t0=0)

x |>
  newick(prune=FALSE,obscure=FALSE) |>
  parse_newick(t0=0)

x |>
  newick(extended=FALSE) |>
  parse_newick(t0=0)

"" |> parse_newick() |> newick()

";" |> parse_newick() |> newick()

"((:1,:1):1,:1):1;" |> parse_newick() |> newick()
```

phylopomp *Phylodynamics for POMP models*

Description

Simulation and inference of Markov genealogy processes.

Author(s)

Aaron A. King, Qianying Lin

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:10.1016/j.tpb.2021.11.003.

plot *Fancy tree plotter*

Description

Plots a genealogical tree.

Usage

```
## S3 method for class 'gpgen'
plot(
  x,
  ...,
  prune = TRUE,
  obscure = TRUE,
  points = FALSE,
  ladderize = TRUE,
  palette = scales::hue_pal(l = 30, h = c(220, 580))
)
```

Arguments

x	object of class ‘gpgen’
...	plot passes extra arguments to theme.
prune	logical; prune the genealogy?
obscure	logical; obscure the demes?

points	logical; show nodes and tips?
ladderize	logical; ladderize?
palette	color palette for indicating demes. This can be furnished either as a function or a vector of colors. If this is a function, it should take a single integer argument, the number of colors required. If it is a vector, it should have at least as many elements as there are demes in the genealogy.

Details

Tree-plotting in **phylopomp** depends on the **ggtree** package.

Value

A printable ggplot object.

References

G. Yu, D. K. Smith, H. Zhu, Y. Guan, T. T. Y. Lam. **ggtree**: an R package for visualization and annotation of phylogenetic trees with their covariates and other associated data. *Methods in Ecology and Evolution* **8**, 28–36, 2017. doi:10.1111/2041210X.12628.

Examples

```
simulate("SEIR",Beta=2,sigma=2,gamma=1,psi=2,S0=1,I0=0.01,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()

runSEIR(Beta=3,gamma=1,psi=2,S0=1,I0=0.2,R0=0,time=5,t0=-1) |>
  plot(points=TRUE,obscure=FALSE)

runSEIR(Beta=3,gamma=0.1,psi=0.2,S0=1,I0=0.05,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))

simulate("SEIR",sigma=1,omega=1,time=20,I0=0.04) |> plot(obscure=FALSE)

simulate("SEIR",sigma=1,omega=1,time=20,I0=0.04) |>
  lineages(obscure=FALSE) |>
  plot()
simulate("LBDP",time=4) |> plot(points=TRUE)

simulate("LBDP",lambda=2,mu=1,psi=3,n0=1,time=1) |>
  simulate(time=10,lambda=1) |>
  plot()

simulate("LBDP",time=4) |>
  lineages() |>
  plot()

runS2I2R2(
  time=30,
```

```
  iota2=0.01,Beta12=0.1,
  I1_0=0,I2_0=0,
  psi1=10,psi2=10,
  omega1=0.2,
  b2=0.02,d2=0.02
) |>
  freeze(seed=847110120) -> x

plot_grid(
  x |>
    plot(
      obscure=FALSE,
      palette=c("#000000ff","#440154ff","#21908cff","#cccccccff"),
      legend.position="inside",
      legend.position.inside=c(0.7,0.7)
    ),
  x |>
    lineages(obscure=FALSE) |>
    plot(
      palette=c("#000000ff","#440154ff","#21908cff","#cccccccff"),
      legend.position="none"
    ),
  ncol=1,
  align="hv",
  axis="b"
)
## Not run:

  library(ggplot2)
  library(phylopomp)
  times <- seq(from=0,to=8,by=0.1)[-1]

  png_files <- sprintf(
    file.path(tempdir(),"frame%05d.png"),
    seq_len(2*length(times))
  )

  pb <- utils::txtProgressBar(0,2*length(times),0,style=3)
  x <- simulate("SIIR",time=0,Beta1=5,Beta2=10,gamma=1,omega=0.5,
    psi1=0.2,psi2=0.1,sigma12=1,sigma21=1,pop=205,S_0=200,I1_0=3,I2_0=2)

  img <- 1
  for (k in seq.int(from=1,to=length(times),by=1)) {
    x <- simulate(x,time=times[k])
    ggsave(
      filename=png_files[img],
      plot=plot(
        x,
        points=FALSE, prune=FALSE, obscure=FALSE,
        ladderize=FALSE,
        palette=c("#000000","#ffcb05","#dddddd"),
        axis.line=element_line(color="white"),
        axis.ticks=element_line(color="white"),
```

```
      axis.text=element_blank(),
      plot.background=element_rect(fill=NA,color=NA),
      panel.background=element_rect(fill=NA,color=NA),
      legend.position="none"
    )+
      expand_limits(x=c(0,range(times))),
    device="png",dpi=300,
    height=2,width=3,units="in"
  )
  setTxtProgressBar(pb,img)
  img <- img+1
}

for (k in seq.int(from=length(times),to=1,by=-1)) {
  x <- curtail(x,time=times[k])
  ggsave(
    filename=png_files[img],
    plot=plot(
      x,
      points=FALSE, prune=FALSE, obscure=FALSE,
      ladderize=FALSE,
      palette=c("#000000","#ffcb05","#dddddd"),
      axis.line=element_line(color="white"),
      axis.ticks=element_line(color="white"),
      axis.text=element_blank(),
      plot.background=element_rect(fill=NA,color=NA),
      panel.background=element_rect(fill=NA,color=NA),
      legend.position="none"
    )+
      expand_limits(x=c(0,range(times))),
    device="png",dpi=300,
    height=2,width=3,units="in"
  )
  setTxtProgressBar(pb,img)
  img <- img+1
}

library(gifski)
gif_file <- "movie1.gif"
gifski(png_files,gif_file,delay=0.08,loop=TRUE)
unlink(png_files)


## End(Not run)
```

`reexports` *Objects exported from other packages*

Description

These objects are imported from other packages. Follow the links below to see their documentation.

cowplot plot_grid()

foreach %dopar%, foreach(), registerDoSEQ()

grid viewport()

pomp bake(), freeze(), stew()

yaml as.yaml(), read_yaml()

---

s2i2r *Two-host infection model with waning, immigration, and demography.*

Description

The population is structured by infection progression and host species.

Usage

```
runS2I2R2(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueS2I2R2(
  object,
  time,
  Beta11 = NA,
  Beta12 = NA,
```

```
  Beta22 = NA,
  gamma1 = NA,
  gamma2 = NA,
  psi1 = NA,
  psi2 = NA,
  omega1 = NA,
  omega2 = NA,
  b1 = NA,
  b2 = NA,
  d1 = NA,
  d2 = NA,
  iota1 = NA,
  iota2 = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta11	transmission rate within species 1
Beta12	transmission from species 2 to species 1
Beta22	transmission rate within species 2
gamma1	species 1 recovery rate
gamma2	species 2 recovery rate
psi1	per capita sampling rate for species 1
psi2	per capita sampling rate for species 2
omega1	rate of waning of immunity for species 1
omega2	rate of waning of immunity for species 2
b1	per capita birth rate for species 1
b2	per capita birth rate for species 2
d1	per capita death rate for species 1
d2	per capita death rate for species 2
iota1	imported infections for species 1
iota2	imported infections for species 2
S1_0	initial size of species 1 susceptible population
S2_0	initial size of species 2 susceptible population
I1_0	initial size of species 1 infected population
I2_0	initial size of species 2 infected population
R1_0	initial size of species 1 immune population
R2_0	initial size of species 2 immune population
object	a previously computed simulation

Value

runS2I2R2 and continueS2I2R2 return objects of class 'gpsim' with 'model' attribute "S2I2R2".

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, seir, si2r, siir, simulate(), sir, strains, twospecies, twoundead

---

seir *Classical susceptible-exposed-infected-recovered model*

Description

The population is structured by infection progression. There is an incubation period that must pass before an infected host becomes infectious. The duration of this period is $1/\sigma$. The infectious period is $1/\gamma$.

Usage

```
runSEIR(
  time,
  t0 = 0,
  Beta = 4,
  sigma = 1,
  gamma = 1,
  psi = 1,
  chi = 0,
  omega = 0,
  pop = 100,
  S0 = 0.9,
  E0 = 0.05,
  I0 = 0.05,
  R0 = 0
)

runSEIRS(
  time,
  t0 = 0,
  Beta = 4,
  sigma = 1,
  gamma = 1,
  psi = 1,
  chi = 0,
  omega = 0,
  pop = 100,
  S0 = 0.9,
  E0 = 0.05,
```

```
  I0 = 0.05,
  R0 = 0
)

continueSEIR(
  object,
  time,
  Beta = NA,
  sigma = NA,
  gamma = NA,
  psi = NA,
  chi = NA,
  omega = NA
)

continueSEIRS(
  object,
  time,
  Beta = NA,
  sigma = NA,
  gamma = NA,
  psi = NA,
  chi = NA,
  omega = NA
)

seirs_pomp(x, Beta, sigma, gamma, psi, chi = 0, omega = 0, S0, E0, I0, R0, pop)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta	transmission rate
sigma	progression rate
gamma	recovery rate
psi	per capita non-destructive sampling rate
chi	per capita destructive sampling rate
omega	rate of waning of immunity
pop	host population size
S0	initial fraction of population susceptible to infection
E0	initial fraction of population in exposed class
I0	initial fraction of population in infectious class
R0	initial fraction of population immune to infection
object	a previously computed simulation
x	genealogy in **phylopomp** format.

Details

seirs_pomp constructs a 'pomp' object containing a given set of data and an SEIRS model.

Value

runSEIR and continueSEIR return objects of class 'gpsim' with 'model' attribute "SEIR".

seirs_pomp returns a 'pomp' object.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, s2i2r2, si2r, siir, simulate(), sir, strains, twospecies, twoundead

Examples

```
simulate("SEIR",Beta=2,sigma=2,gamma=1,psi=2,S0=1,I0=0.01,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()

runSEIR(Beta=3,gamma=1,psi=2,S0=1,I0=0.2,R0=0,time=5,t0=-1) |>
  plot(points=TRUE,obscure=FALSE)

runSEIR(Beta=3,gamma=0.1,psi=0.2,S0=1,I0=0.05,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))

simulate("SEIR",sigma=1,omega=1,time=20,I0=0.04) |> plot(obscure=FALSE)

simulate("SEIR",sigma=1,omega=1,time=20,I0=0.04) |>
  lineages(obscure=FALSE) |>
  plot()
```

---

si2r *Two-deme model of superspreading*

Description

Deme L consists of "low-rate spreaders" that transmit at rate Beta. Deme H consists of "super-spreaders" who transmit at a higher rate kappa*Beta. Sampling is destructive.

Usage

```
runSI2R(
  time,
  t0 = 0,
  Beta = 5,
  kappa = 2,
  gamma = 1,
  omega = 0,
  chi = 1,
  etaL = 1,
  etaH = 3,
  pop = 500,
  S0 = 0.98,
  IL0 = 0.02,
  IH0 = 0,
  R0 = 0
)

continueSI2R(
  object,
  time,
  Beta = NA,
  kappa = NA,
  gamma = NA,
  omega = NA,
  chi = NA,
  etaL = NA,
  etaH = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta	transmission rate
kappa	super-spreading transmission factor
gamma	recovery rate
omega	rate of waning of immunity
chi	(destructive) sampling rate
etaL	rate of transition from low-spreading to super-spreading behavior
etaH	rate of transition from super-spreading to low-spreading behavior
pop	population size
S0	initial fraction of susceptible population
IL0	initial fraction of low-spreading population
IH0	initial fraction of super-spreading population

R0	initial fraction of immune population
object	a previously computed simulation

Value

runSI2R and continueSI2R return objects of class 'gpsim' with 'model' attribute "SI2R".

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, s2i2r2, seir, siir, simulate(), sir, strains, twospecies, twoundead

---

siir *Two-strain SIR model*

Description

Two distinct pathogen strains compete for susceptibles.

Usage

```
runSIIR(
  time,
  t0 = 0,
  Beta1 = 5,
  Beta2 = 5,
  gamma = 1,
  psi1 = 1,
  psi2 = 0,
  sigma12 = 0,
  sigma21 = 0,
  omega = 0,
  pop = 500,
  S_0 = 0.96,
  I1_0 = 0.02,
  I2_0 = 0.02,
  R_0 = 0
)

continueSIIR(
  object,
  time,
  Beta1 = NA,
  Beta2 = NA,
  gamma = NA,
  psi1 = NA,
  psi2 = NA,
```

```
  sigma12 = NA,
  sigma21 = NA,
  omega = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta1	transmission rate for strain 1
Beta2	transmission rate for strain 2
gamma	recovery rate
psi1	sampling rate for deme 1
psi2	sampling rate for deme 2
sigma12	rate of movement from deme 1 to deme 2
sigma21	rate of movement from deme 2 to deme 1
omega	rate of loss of immunity
pop	population size
S_0	initial fraction of susceptible population
I1_0	initial fraction of deme-1 infected population
I2_0	initial fraction of deme-2 infected population
R_0	initial fraction of immune population
object	a previously computed simulation

Value

runSIIR and continueSIIR return objects of class 'gpsim' with 'model' attribute "SIIR".

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, s2i2r2, seir, si2r, simulate(), sir, strains, twospecies, twoundead

---

simulate *simulate*

Description

Simulate Markov genealogy processes

Usage

```
simulate(object, ...)

## Default S3 method:
simulate(object, ...)

## S3 method for class 'character'
simulate(object, time, ...)

## S3 method for class 'gpsim'
simulate(object, time, ...)
```

Arguments

`object`	either the name of the model to simulate *or* a previously computed 'gpsim' object
`...`	additional arguments to the model-specific simulation functions
`time`	end timepoint of simulation

Details

When `object` is of class 'gpsim', i.e., the result of a genealogy-process simulation, `simulate` acts to continue the simulation to a later timepoint. Note that, one cannot change initial conditions or `t0` when continuing a simulation.

Value

An object of 'gpsim' class.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:10.1016/j.tpb.2021.11.003.

See Also

More example genealogy processes: `bdei`, `bdss`, `lbdp`, `mers`, `moran`, `s2i2r2`, `seir`, `si2r`, `siir`, `sir`, `strains`, `twospecies`, `twoundead`

sir *Classical susceptible-infected-recovered model*

Description

A single, unstructured population of hosts.

Usage

```
runSIR(
  time,
  t0 = 0,
  Beta = 4,
  gamma = 1,
  psi = 1,
  omega = 0,
  pop = 100,
  S0 = 0.95,
  I0 = 0.05,
  R0 = 0
)

continueSIR(object, time, Beta = NA, gamma = NA, psi = NA, omega = NA)

runSIRS(
  time,
  t0 = 0,
  Beta = 4,
  gamma = 1,
  psi = 1,
  omega = 0,
  pop = 100,
  S0 = 0.95,
  I0 = 0.05,
  R0 = 0
)

continueSIRS(object, time, Beta = NA, gamma = NA, psi = NA, omega = NA)

sir_pomp(x, Beta, gamma, psi, omega = 0, S0, I0, R0, pop)

sirs_pomp(x, Beta, gamma, psi, omega = 0, S0, I0, R0, pop)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation

`Beta`	transmission rate
`gamma`	recovery rate
`psi`	per capita (nondestructive) sampling rate
`omega`	rate of waning of immunity
`pop`	size of population
`S0`	initial size of susceptible population
`I0`	initial size of infected population
`R0`	initial size of immune population
`object`	a previously computed simulation
`x`	genealogy in **phylopomp** format (i.e., an object that inherits from 'gpgen').

Details

`sir_pomp` constructs a 'pomp' object containing a given set of data and a SIR model.

Value

`runSIR` and `continueSIR` return objects of class 'gpsim' with 'model' attribute "SIR".

`sir_pomp` and `sirs_pomp` return 'pomp' objects.

References

A. A. King, Q. Lin, and E. L. Ionides. Exact phylodynamic likelihood via structured Markov genealogy processes. *arXiv* 2405.17032, 2024. doi:10.48550/arxiv.2405.17032.

A. A. King, Q. Lin, and E. L. Ionides. Markov genealogy processes. *Theoretical Population Biology* **143**, 77–91, 2022. doi:10.1016/j.tpb.2021.11.003.

See Also

More example genealogy processes: `bdei`, `bdss`, `lbdp`, `mers`, `moran`, `s2i2r2`, `seir`, `si2r`, `siir`, `simulate()`, `strains`, `twospecies`, `twoundead`

Examples

```
simulate("SIR",Beta=2,gamma=1,psi=2,pop=1000,I0=0.005,time=5) |>
  simulate(Beta=5,gamma=2,time=10,psi=3) |>
  plot()

runSIR(Beta=3,gamma=1,psi=2,pop=25,S0=0.8,I0=0.2,R0=0,time=5,t0=-1) |>
  plot(points=TRUE)

runSIR(Beta=3,gamma=0.1,psi=0.2,pop=100,S0=1,I0=0.05,R0=0,time=2,t0=0) -> x
plot_grid(plotlist=list(plot(x,points=TRUE),diagram(x)),
  ncol=1,rel_heights=c(4,1))

simulate("SIRS",omega=1,time=20,I0=0.05) |> plot()
simulate("SIRS",omega=1,time=20,I0=0.05) |> lineages() |> plot()
```

strains *Three strains compete for a single susceptible pool.*

Description

The three demes are three distinct pathogen strains that compete for susceptibles. Sampling is assumed destructive, i.e., sampled individuals are removed from the infectious pool.

Usage

```
runStrains(
  time,
  t0 = 0,
  Beta1 = 5/7,
  Beta2 = 5/7,
  Beta3 = 5/7,
  gamma = 1/7,
  chi = 0.002,
  pop = 1e+06,
  S_0 = 0.9,
  I1_0 = 0.003,
  I2_0 = 0.003,
  I3_0 = 0.003,
  R_0 = 0.1
)

continueStrains(
  object,
  time,
  Beta1 = NA,
  Beta2 = NA,
  Beta3 = NA,
  gamma = NA,
  chi = NA
)

strains_pomp(
  x,
  Beta1,
  Beta2,
  Beta3,
  gamma,
  chi,
  pop,
  S_0,
  I1_0,
  I2_0,
```

```
  I3_0,
  R_0
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta1	transmission rate for strain 1
Beta2	transmission rate for strain 2
Beta3	transmission rate for strain 3
gamma	recovery rate
chi	(destructive) sampling rate
pop	population size
S_0	initial susceptible fraction
I1_0	initial fraction of population infected by strain 1
I2_0	initial fraction of population infected by strain 2
I3_0	initial fraction of population infected by strain 2
R_0	initial fraction of population immune
object	a previously computed simulation
x	genealogy in **phylopomp** format (i.e., an object that inherits from 'gpgen').

Details

strains_pomp constructs a 'pomp' object containing a given set of data and the Strains model.

Value

runStrains and continueStrains return objects of class 'gpsim' with 'model' attribute "Strains".

strains_pomp returns a 'pomp' object.

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, s2i2r2, seir, si2r, siir, simulate(), sir, twospecies, twoundead

`twospecies` *Two-host infection model with waning, immigration, demography, and spillover. Hosts are culled upon sampling with a given probability.*

Description

The population is structured by infection progression and host species.

Usage

```
runTwoSpecies(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta21 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  c1 = 1,
  c2 = 1,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueTwoSpecies(
  object,
  time,
  Beta11 = NA,
  Beta12 = NA,
  Beta21 = NA,
  Beta22 = NA,
  gamma1 = NA,
```

```
  gamma2 = NA,
  psi1 = NA,
  psi2 = NA,
  c1 = NA,
  c2 = NA,
  omega1 = NA,
  omega2 = NA,
  b1 = NA,
  b2 = NA,
  d1 = NA,
  d2 = NA,
  iota1 = NA,
  iota2 = NA
)

twospecies_pomp(
  x,
  Beta11,
  Beta12,
  Beta21,
  Beta22,
  gamma1,
  gamma2,
  psi1,
  psi2,
  c1,
  c2,
  omega1,
  omega2,
  b1,
  b2,
  d1,
  d2,
  S1_0,
  S2_0,
  I1_0,
  I2_0,
  R1_0,
  R2_0
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta11	transmission rate within species 1
Beta12	transmission from species 2 to species 1
Beta21	transmission from species 1 to species 2

Beta22	transmission rate within species 2
gamma1	species 1 recovery rate
gamma2	species 2 recovery rate
psi1	per capita sampling rate for species 1
psi2	per capita sampling rate for species 2
c1	probability that a sampled (positive) host of species 1 is culled
c2	probability that a sampled (positive) host of species 2 is culled
omega1	rate of waning of immunity for species 1
omega2	rate of waning of immunity for species 2
b1	per capita birth rate for species 1
b2	per capita birth rate for species 2
d1	per capita death rate for species 1
d2	per capita death rate for species 2
iota1	imported infections for species 1
iota2	imported infections for species 2
S1_0	initial size of species 1 susceptible population
S2_0	initial size of species 2 susceptible population
I1_0	initial size of species 1 infected population
I2_0	initial size of species 2 infected population
R1_0	initial size of species 1 immune population
R2_0	initial size of species 2 immune population
object	a previously computed simulation
x	genealogy in **phylopomp** format.

Details

`twospecies_pomp` constructs a 'pomp' object containing a given set of data and a TwoSpecies model. Note that, for the moment, `twospecies_pomp` assumes that there is no importation of infection into the populations (i.e., `iota1 = iota2 = 0`).

Value

`runTwoSpecies` and `continueTwoSpecies` return objects of class 'gpsim' with 'model' attribute "TwoSpecies".

`twospecies_pomp` returns a 'pomp' object.

See Also

More example genealogy processes: `bdei`, `bdss`, `lbdp`, `mers`, `moran`, `s2i2r2`, `seir`, `si2r`, `siir`, `simulate()`, `sir`, `strains`, `twoundead`

twoundead	*Two-host infection model with waning, immigration, demography, and spillover. Hosts are culled upon sampling with a given probability. This is identical to the TwoSpecies model with the exception that dead lineages are not pruned. Instead, they become *ghosts*.*

Description

The population is structured by infection progression and host species.

Usage

```
runTwoUndead(
  time,
  t0 = 0,
  Beta11 = 4,
  Beta12 = 0,
  Beta21 = 0,
  Beta22 = 4,
  gamma1 = 1,
  gamma2 = 1,
  psi1 = 1,
  psi2 = 0,
  c1 = 1,
  c2 = 1,
  omega1 = 0,
  omega2 = 0,
  b1 = 0,
  b2 = 0,
  d1 = 0,
  d2 = 0,
  iota1 = 0,
  iota2 = 0,
  S1_0 = 100,
  S2_0 = 100,
  I1_0 = 0,
  I2_0 = 10,
  R1_0 = 0,
  R2_0 = 0
)

continueTwoUndead(
  object,
  time,
  Beta11 = NA,
  Beta12 = NA,
  Beta21 = NA,
```

```
  Beta22 = NA,
  gamma1 = NA,
  gamma2 = NA,
  psi1 = NA,
  psi2 = NA,
  c1 = NA,
  c2 = NA,
  omega1 = NA,
  omega2 = NA,
  b1 = NA,
  b2 = NA,
  d1 = NA,
  d2 = NA,
  iota1 = NA,
  iota2 = NA
)
```

Arguments

time	end timepoint of simulation
t0	beginning timepoint of simulation
Beta11	transmission rate within species 1
Beta12	transmission from species 2 to species 1
Beta21	transmission from species 1 to species 2
Beta22	transmission rate within species 2
gamma1	species 1 recovery rate
gamma2	species 2 recovery rate
psi1	per capita sampling rate for species 1
psi2	per capita sampling rate for species 2
c1	probability that a sampled (positive) host of species 1 is culled
c2	probability that a sampled (positive) host of species 2 is culled
omega1	rate of waning of immunity for species 1
omega2	rate of waning of immunity for species 2
b1	per capita birth rate for species 1
b2	per capita birth rate for species 2
d1	per capita death rate for species 1
d2	per capita death rate for species 2
iota1	imported infections for species 1
iota2	imported infections for species 2
S1_0	initial size of species 1 susceptible population
S2_0	initial size of species 2 susceptible population
I1_0	initial size of species 1 infected population

I2_0	initial size of species 2 infected population
R1_0	initial size of species 1 immune population
R2_0	initial size of species 2 immune population
object	a previously computed simulation

Value

runTwoUndead and continueTwoUndead return objects of class 'gpsim' with 'model' attribute "TwoUndead".

See Also

More example genealogy processes: bdei, bdss, lbdp, mers, moran, s2i2r2, seir, si2r, siir, simulate(), sir, strains, twospecies

---

yaml *YAML output*

Description

Human- and machine-readable description.

Usage

```
yaml(object)
```

Arguments

object	gpsim object.

Value

A string in YAML format, with class "gpyaml".

Examples

```
simulate("SIIR",time=1) |> yaml()
```

Index